Automata Theory

Non-Deterministic Finite Automata

CS32022

By Thisara Watawana

12 August 2026

3. Non-Deterministic Finite Automata

Deterministic Finite Automata (DFA)

Everything we have built so far is a Deterministic Finite Automaton (DFA). "Deterministic" means every step of computation is strictly predetermined and predictable.

The 3 Rules of Determinism:

1. Exactly One Choice: For any state q and symbol a, $\delta(q, a)$ outputs exactly one next state.

2. No Missing Outlets: Every state must handle every symbol in Σ (no missing transitions).

3. No Reading ε: The machine cannot change states without consuming an input character.

For any input string w, a DFA executes a single, repeatable sequence of states from start to finish.

3. Non-Deterministic Finite Automata

Deterministic Finite Automata (DFA)

Everything we have built so far is a Deterministic Finite Automaton (DFA). "Deterministic" means every step of computation is strictly predetermined and predictable.

The 3 Rules of Determinism:

1. Exactly One Choice: For any state q and symbol a, $\delta(q, a)$ outputs exactly one next state.

2. No Missing Outlets: Every state must handle every symbol in Σ (no missing transitions).

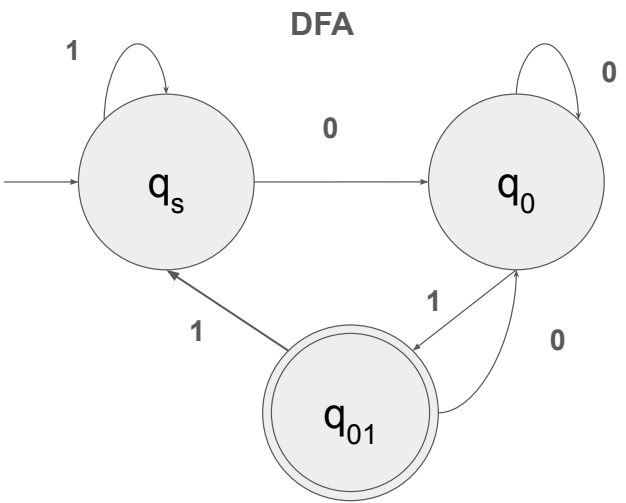
3. No Reading ε: The machine cannot change states without consuming an input character.

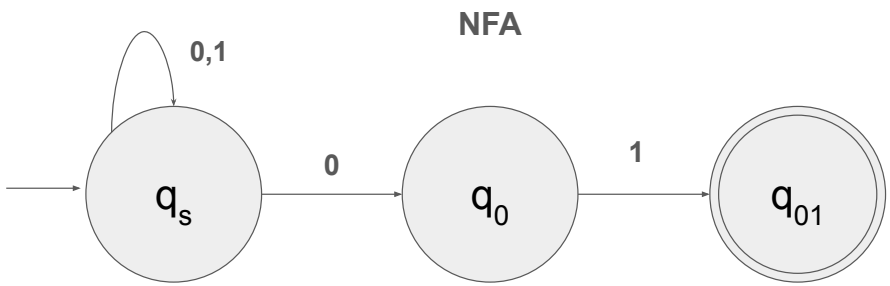
For any input string w, a DFA executes a single, repeatable sequence of states from start to finish.

3. Non-Deterministic Finite Automata

Revisit Exercise 05:

Required tracking full state memory every state needed strict, explicit transitions for both 0 and 1 to handle reset paths (e.g., handling inputs like 00 or 101).





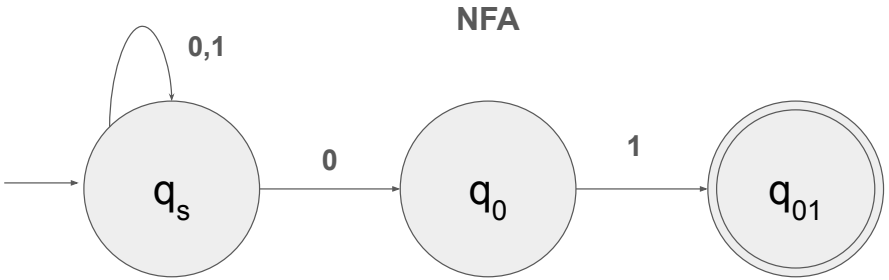
3. Non-Deterministic Finite Automata

Revisit Exercise 05:

q_s stays in the background, looping on every symbol (0, 1).

Whenever a 0 appears, the NFA spawns a new guess path to q_0 to check if this is the start of the final 01 suffix.

If wrong, that path dies in q_0. If right, it lands in q_{01} (Accept).



3. Non-Deterministic Finite Automata

Exercise 01:

Suppose you already know how to build two simple machines:

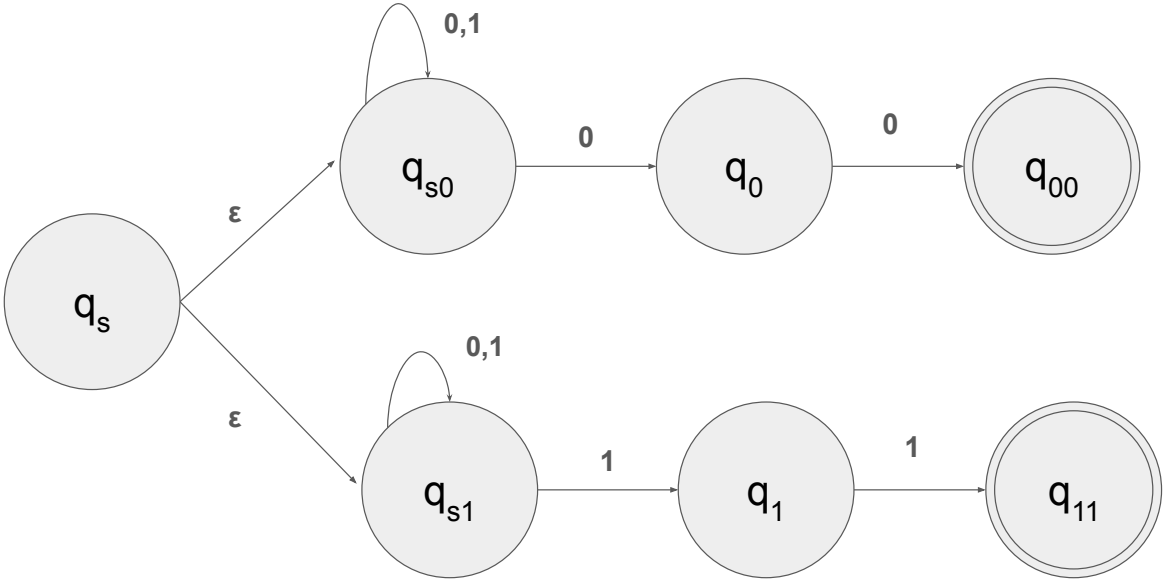
Machine A: Accepts strings containing substring 00.

Machine B: Accepts strings containing substring 11.

Design a single machine over $\Sigma = \{0, 1\}$ that accepts a string if it satisfies Machine A OR Machine B.

3. Non-Deterministic Finite Automata

Exercise 01:



3. Non-Deterministic Finite Automata

ε-Transitions

A transition labeled with ε (the empty string, where |ε| = 0).

- Moving along an ε-arrow consumes 0 input characters from the input string.
- The move happens automatically the instant the machine arrives at the state.
- If a state has outgoing ε-transitions, the machine instantly exists in both the origin state and all target states simultaneously before reading the next character.

Allows a machine to make structural branch choices without eating or losing the first symbol of the input stream.

3. Non-Deterministic Finite Automata

Deterministic vs. Nondeterministic Automata

Feature	DFA (Deterministic)	NFA (Nondeterministic)
Transitions per (State, Symbol)	Exactly **1**	**0, 1, or multiple**
Empty Input Transitions (ε)	Not allowed	**Allowed** (free teleports without reading input)
Computation Path	Single, linear path	Tree of parallel computation paths
Acceptance Condition	Final state $\in F$	**At least one** active path ends in F

3. Non-Deterministic Finite Automata

Formal Definition of an NFA

An NFA is mathematically defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where the components are defined as:

- Q : A finite set of states.
- Σ : A finite alphabet.
- δ : $Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$ is the transition function.
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of accept states.

Critical Differences from DFAs

Unlike deterministic machines, an NFA incorporates two primary mathematical formalisms:

1. ε-Transitions Allowed

The alphabet includes the empty string ε, allowing state changes without consuming input:

$$\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$$

2. Power Set Mapping

The transition function outputs a set of states (elements of the power set $\mathcal{P}(Q)$) rather than a single state:

$$\delta : Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$$

3. Non-Deterministic Finite Automata

Why NFAs Matter?

- Eliminates "reset logic." You design for the success path and let invalid branches crash automatically.
- An NFA with n states can require up to 2^n states to represent as a DFA.
- Makes combining machines (Union, Concatenation, Star) effortless via ε-transitions.
- Real-world pattern matchers (like grep or system regex parsers) construct NFAs under the hood because they are far easier to assemble programmatically.

3. Non-Deterministic Finite Automata

Exercise 02:

Given that the alphabet: $\Sigma = \{0, 1\}$, design an NFA that accepts strings where the 3rd symbol from the right is 1.

Examples: 100 (Accept), 1011 (Reject), 0101 (Accept), 10 (Reject).

3. Non-Deterministic Finite Automata

Exercise 03:

Given that the alphabet: $\Sigma = \{0, 1\}$, Design an NFA that accepts **only** the **exact string 012**.

3. Non-Deterministic Finite Automata

Regular Operations

Let A and B be languages over an alphabet Σ.

01. Union ($A \cup B$):

$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$

Meaning: Accepts strings belonging to either language.

Construction Logic:

- Create a brand-new start state q_0.
- Add ε-transitions from q_0 to the original start state of N_1 and the original start state of N_2.
- Keep all original accept states $F = F_1 \cup F_2$.

3. Non-Deterministic Finite Automata

02. Concatenation ($A \circ B$):

$$A \circ B = \{ xy \mid x \in A \text{ and } y \in B \}$$

Meaning: Accepts strings formed by taking a string from A directly followed by a string from B.

Construction Logic:

- Start state of the new machine is the original start state of N_1.
- Draw an ε-transition from every accept state in F_1 to the start state of N_2.
- Demote F_1: The old accept states of N_1 are no longer accept states!
- The only valid accept states are now F_2 (the accept states of N_2).

3. Non-Deterministic Finite Automata

03. Kleene Star (A^*):

$A^* = \{ x_1x_2...x_k \mid k \geq 0$ and each $x_i \in A \}$

Meaning: Accepts any sequence of zero or more strings glued together from A.

(Note: $k = 0$ means $\varepsilon \in A^*$, so the empty string is always in A^*)

Construction Logic:

- Create a brand-new start state q_0 and make it an accept state (this guarantees $\varepsilon \in A^*$).
- Add an ε-transition from q_0 to N_1's original start state.
- Add feedback ε-transitions from every accept state in F_1 back to N_1's original start state to allow repeated loops.

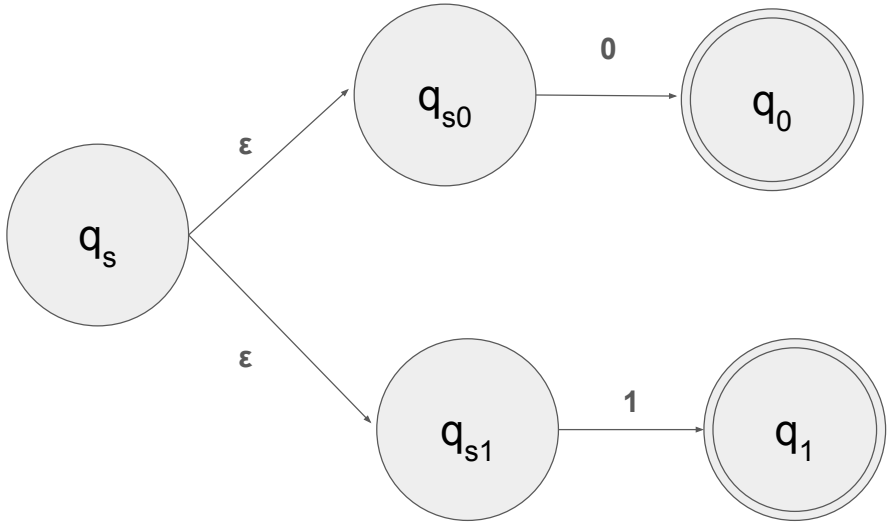
3. Non-Deterministic Finite Automata

Exercise 04:

Given Automata:

- N_1 accepts $L_1 = \{0\}$
- N_2 accepts $L_2 = \{1\}$

Construct an NFA for $L_1 \cup L_2 = \{0, 1\}$.

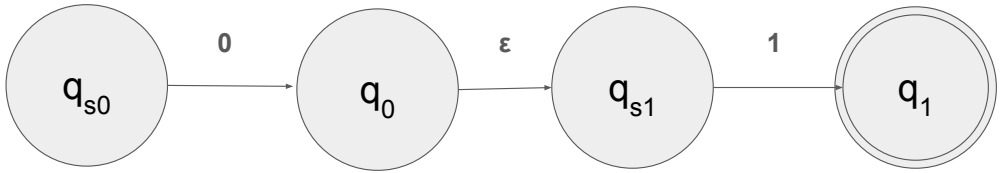


3. Non-Deterministic Finite Automata

Exercise 05:

Given Automata:

- *N*1 accepts *L*1 = {0}
- *N*2 accepts *L*2 = {1}



Construct an NFA for *L*1 ◦ *L*2 = {01}.

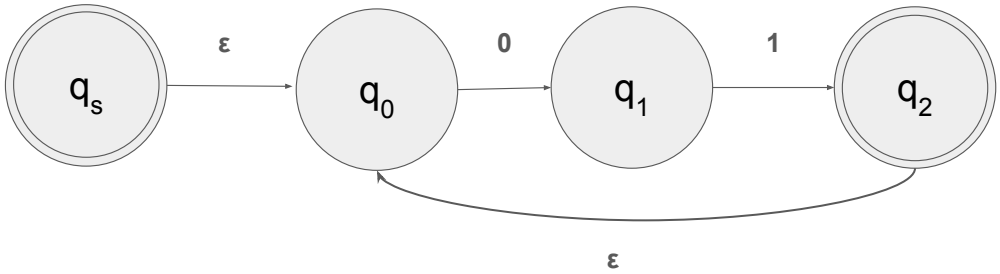
3. Non-Deterministic Finite Automata

Exercise 05:

Given Automaton: N_1 accepts $L_1 = \{01\}$

Construct an NFA for

$L_1^* = \{01\}^* = \{\varepsilon, 01, 0101, 010101, ...\}$



Questions?

thisaraw@boffin.lk